

Student Feedback Survey



Machine Learning
Algorithms and
Applications

<https://go.blueja.io/KxnqfkWtL0i0FXpAAOwj5A>

- Completely anonymous
- All feedback are welcomed

The screenshot shows the login interface for the UTS online Student Feedback Survey (SFS). At the top, there's a banner with the survey title and a photo of students. Below the banner, a status bar shows 'Open online surveys: 42' and 'Current participation: 14.94%'. The main heading is 'Welcome to the UTS online Student Feedback Survey (SFS)'. It includes instructions for students and staff to enter their UTS ID and password. There are two login sections: one for 'STUDENT' and one for 'STAFF', each with input fields for 'UTS ID' and 'UTS password' and a 'GO' button. A 'SFS Privacy Notice' link is at the bottom left. The footer contains 'UTSOnline', 'One Stop Student Admin', and the version 'v2.4.1'.

EARLY FEEDBACK SURVEY

STUDENT FEEDBACK SURVEY

Open online surveys: 42 FASS FEET BUS DSH TDI DAB HEALTH
Current participation: 14.94%

Tue, 7 Mar 2023 8:36 am AET
students online 0

Welcome to the UTS online Student Feedback Survey (SFS)

Tell us about your learning experience at UTS.
Inside are currently open surveys waiting to be completed. You can also look at past results.
Your feedback is confidential. When you submit the SFS, your student ID registers your participation and enters you into the prize draw.
Two related surveys are now conducted for every subject each session:
Early Feedback Survey (EFS) - a short survey run during week 4 for you to provide an early indication of how you feel about each subject.
Student Feedback Survey (SFS) - run at session's end for you to provide feedback on your learning experience in each subject.
Teaching staff will be able to see aggregated results only. For the SFS, results are released after the exam marks have been finalised.
What are you waiting for ...
... please login and give us your constructive feedback!

SFS Privacy Notice

UTSOnline One Stop Student Admin v2.4.1

Machine Learning Algorithms and Applications

Lecture 9

(Block 4)

Plan for Today



Intro to Neural Networks



Back Propagation



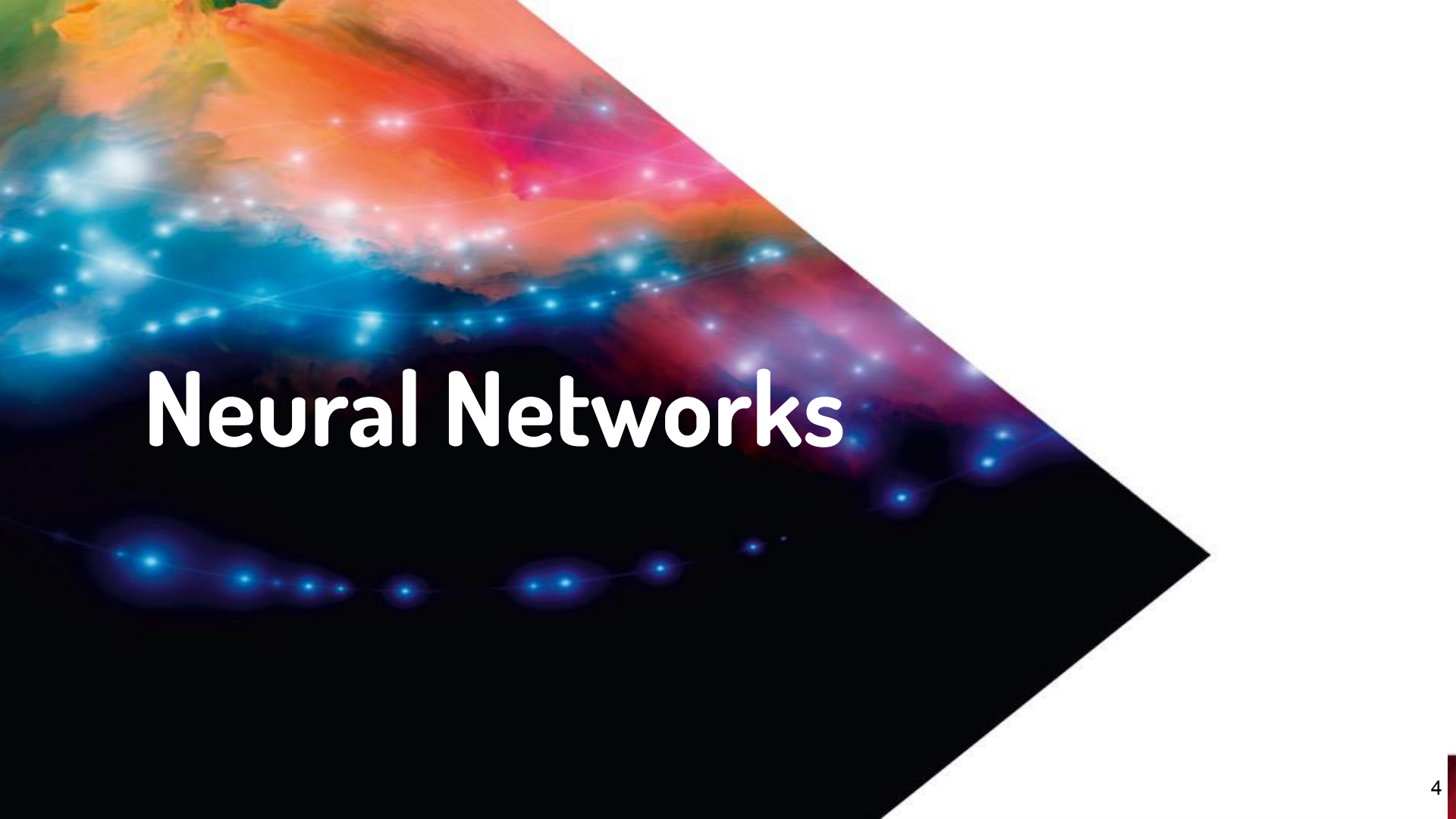
Forward Propagation



**Multi-Layer Perceptron with
Sklearn**



Lab



Neural Networks

Neural Networks



Applications of Neural Networks



01

**Handwriting
Recognition**



03

Image Compression



02

**Traveling Salesman
Problem**

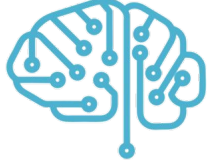


04

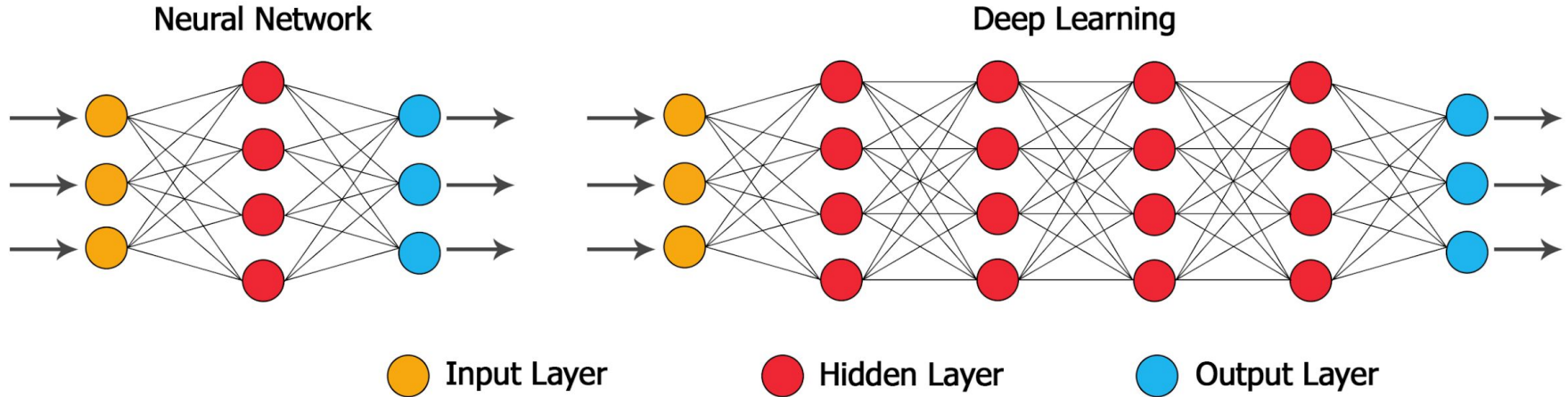
**Stock Exchange
Prediction**



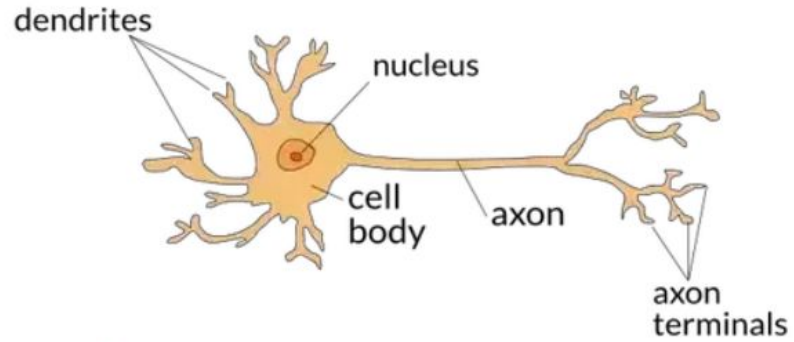
AI, ML, DL

		
Artificial Intelligence	Machine Learning	Deep Learning
Artificial intelligence originated around 1950s.	Machine learning originated around 1960s.	Deep learning originated around 1970s.
AI represents simulated intelligence in machines.	Machine Learning is the practice of getting machines to make decisions without being programmed.	Deep Learning is the process of using Artificial Neural Networks to solve complex problems.
AI is a subset of Data Science.	Machine learning is a subset of AI & Data Science	Deep learning is a subset of Machine learning, AI & Data Science.
Aim is to build machines which are capable of thinking like humans.	Aim is to make machines learn through data so that they can solve problems.	Aim is to build neural networks that automatically discover patterns for feature detection.

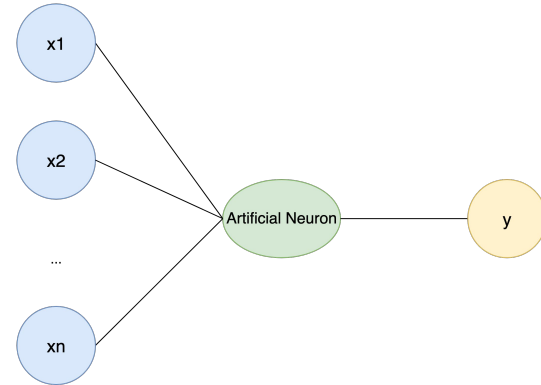
NeuralNets vs Deep Learning



Neurons

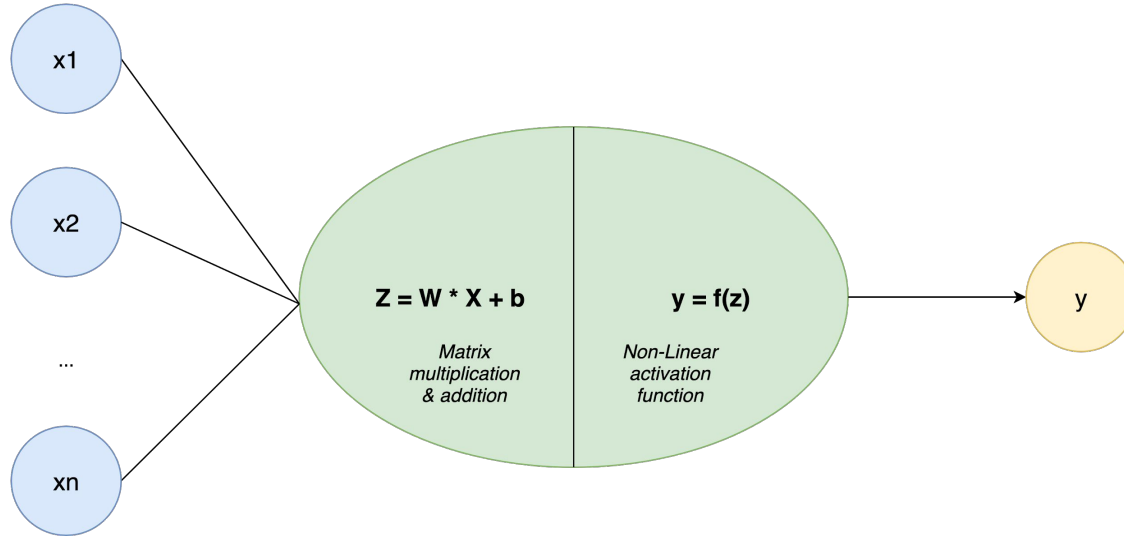


Real Neuron



Artificial Neuron

Inside an Artificial Neuron



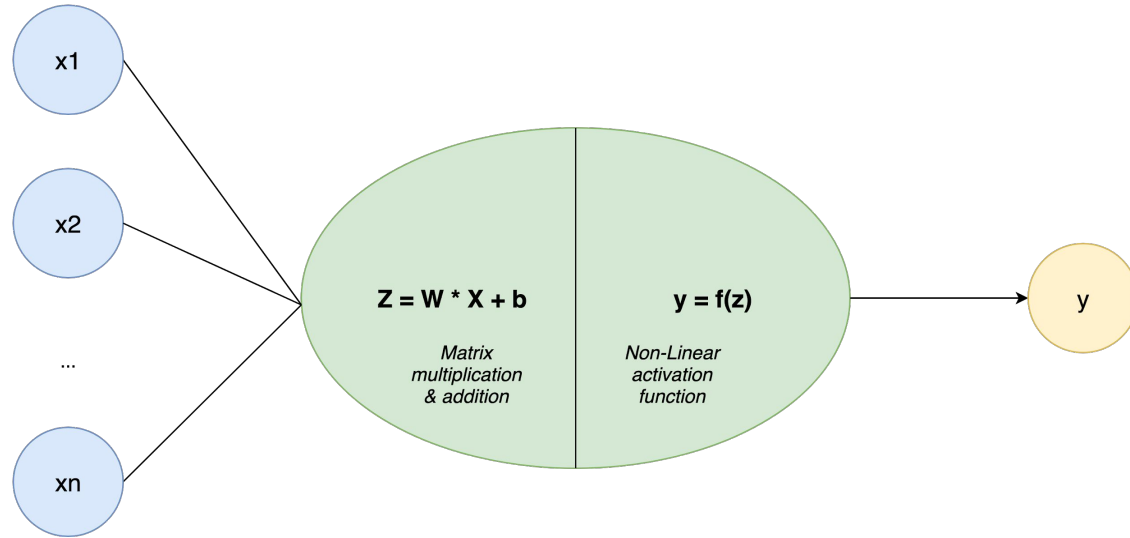
- X is the input matrix, which is composed of values: x_1 , x_2 and x_3
- W , the weight matrix, will be composed values: w_1 , w_2 and w_3
- b is the bias (constant),
- f is the activation function.

$[x_1, x_2, x_3]$

$$\begin{aligned} Z &= W * X + b \\ &= x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + b \end{aligned}$$

$$\begin{aligned} y &= f(Z) \\ &= f(x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + x_4 * w_4 + b) \end{aligned}$$

Linear Regression with NeuralNets

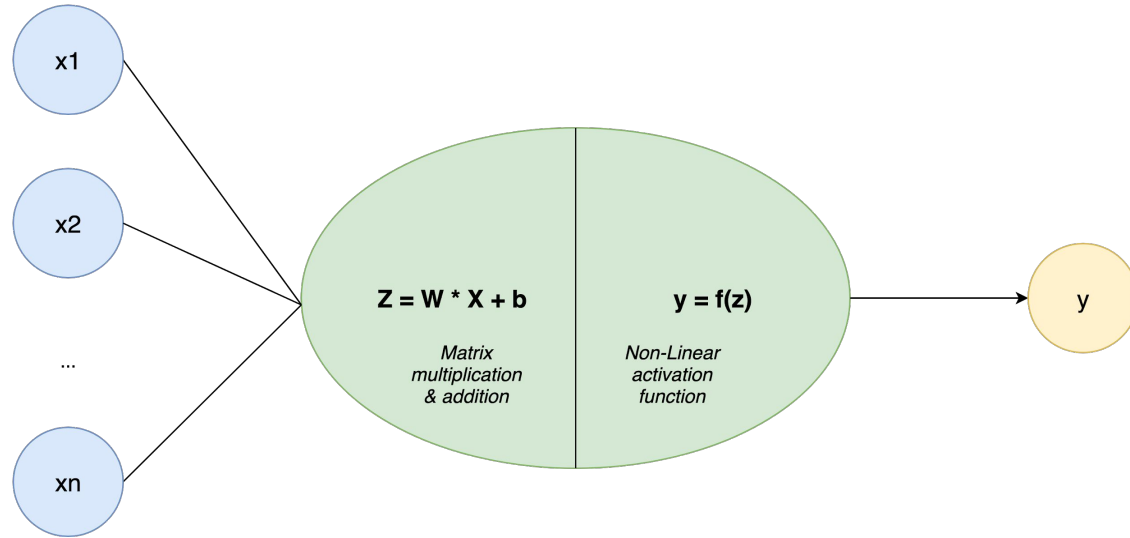


$$X \in R^N$$

$$f(x) = \text{identity}(x) = x$$

$$y \in R$$

Logistic Regression with NeuralNets



$$X \in R^N$$

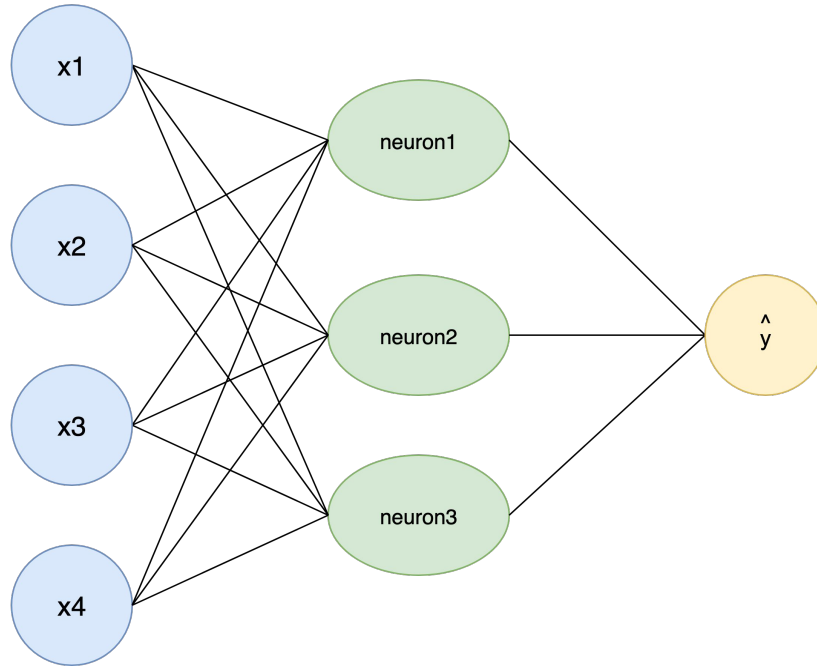
$$f(x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

$$0 \leq y \leq 1$$



Forward Propagation

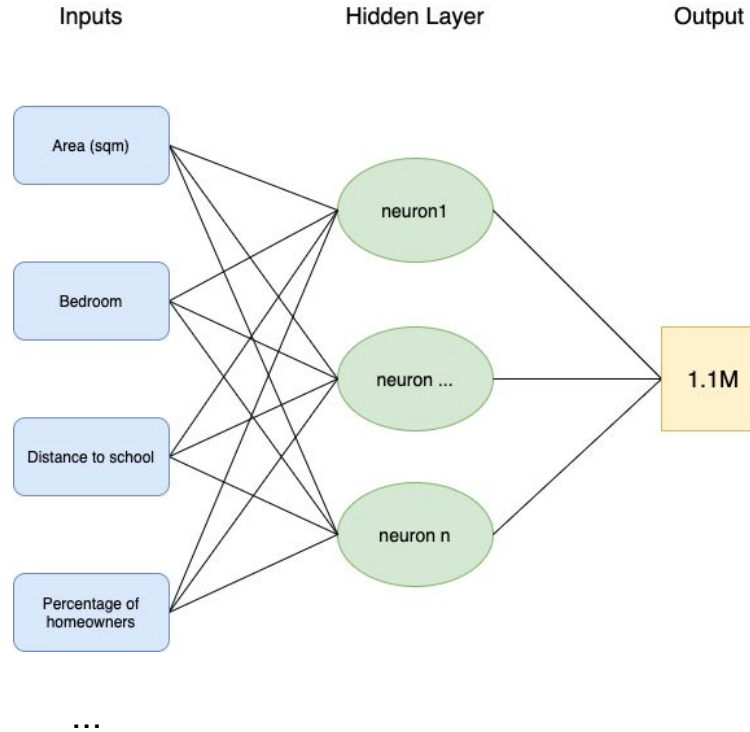
Layers



- Rather than just a single neuron we can stack multiple ones into a fully connected layer before making a prediction
- A Neuralnet is defined by its:
 - Input layer: X
 - Output layer: y
 - Hidden layer: neurons

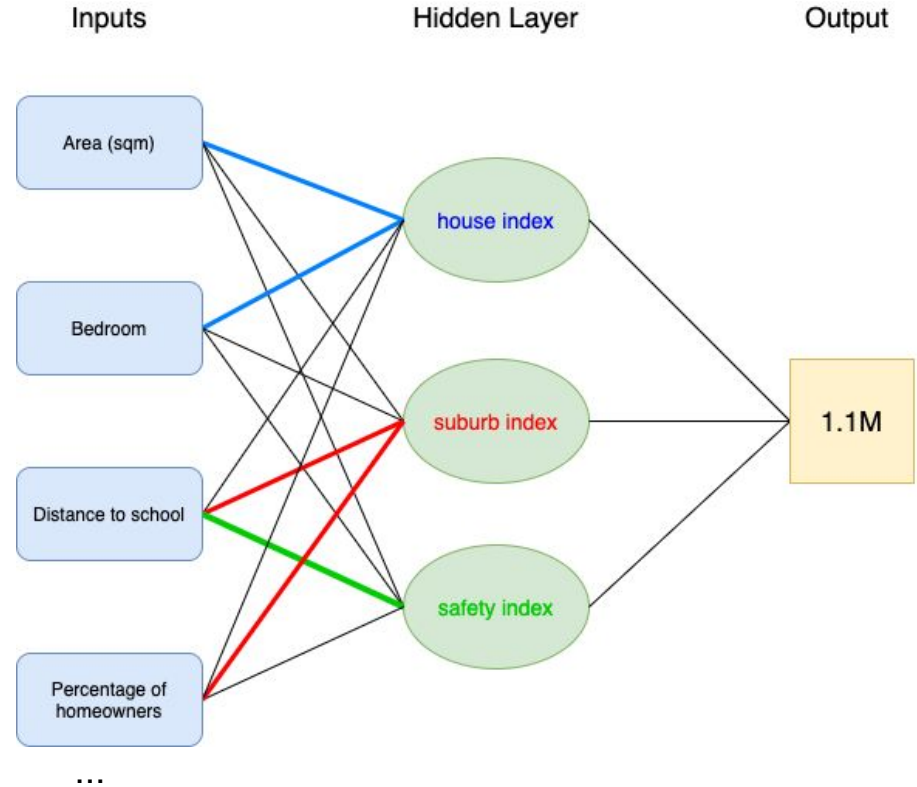
Example

- Predicting the price of real estate property
- Features
 - Attributes of house
 - Specificities
 - Suburb information
 - Last auctions results

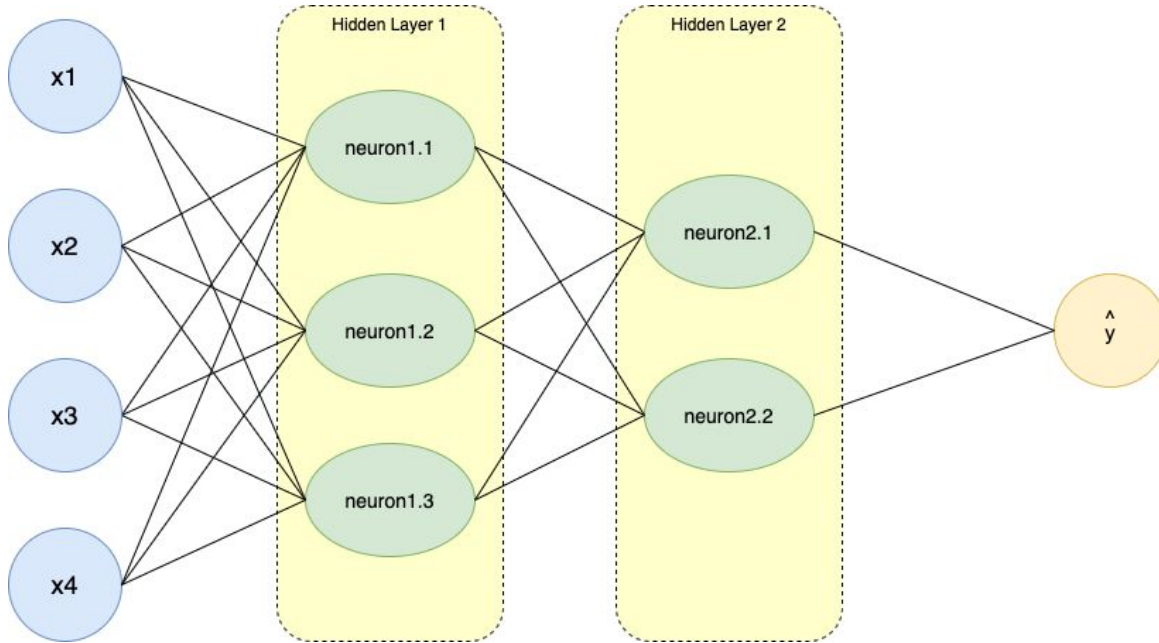


Example

- Each neuron will use some specific inputs to create a new feature (think of it as an index)
- Each neuron will be different from each other and find different composite feature (different weights)
- The output layer will use these composite feature to make prediction

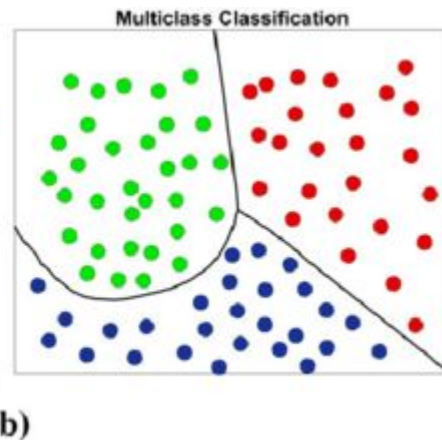
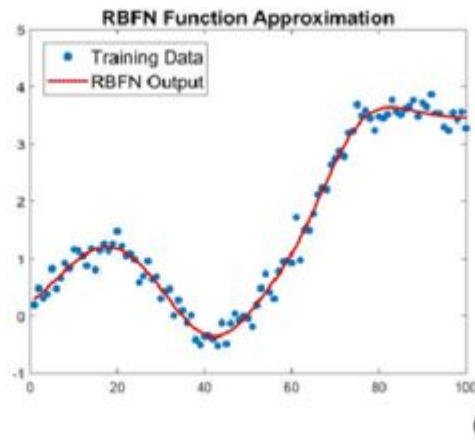
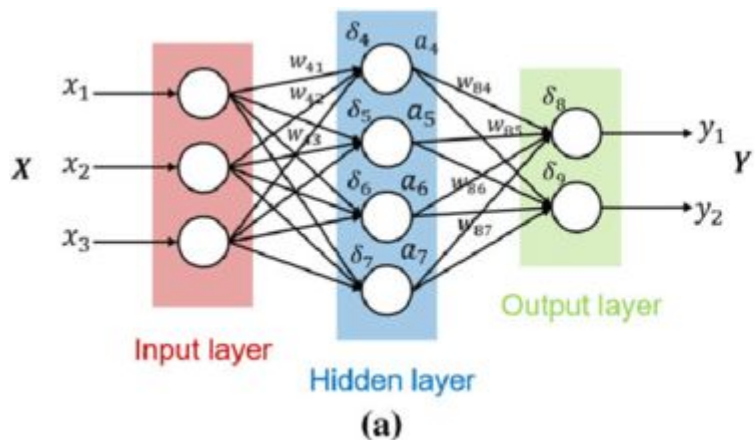


Deep Learning



- More than 1 hidden layer = Deep Learning
- Each hidden layer will use the composite feature of the previous layer to create new ones (more and more complex and abstract)
- Hidden layers close to the output layer are more specific to the predictions

NeuralNets, function approximator



Activation Functions

- Activation functions are used to introduce non-linearity in the function approximation (fit complex patterns from data)
- Need to choose the activation function for:
 - Hidden layer (ReLU in general)
 - Output layer (depends on the problem space)

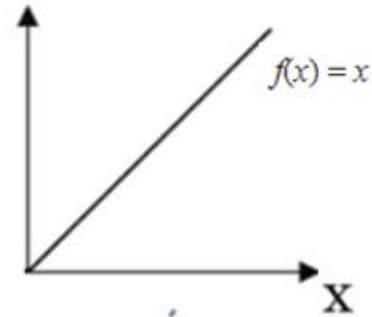
Name	Plot	Equation
Identity		$f(x) = x$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$
ArcTan		$f(x) = \tan^{-1}(x)$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$

Identity Function

$$f(x) = \text{identity}(x) = x$$

- No transformation performed on data
- Output is continuous between $[-\infty, +\infty]$
- Mainly used for the output layer in regression problems

Identity Function



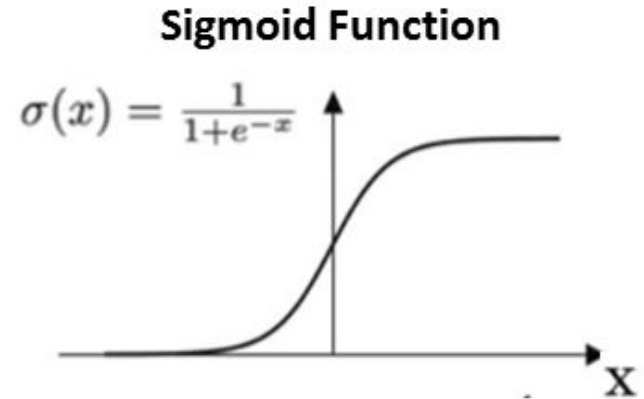
Sigmoid Function

$$f(x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

- Non-linear transformation performed on data
- Output is continuous between

$[0, 1]$

- Mainly used for the output layer in binary classification problems and in RNN for hidden layers



ReLu Function

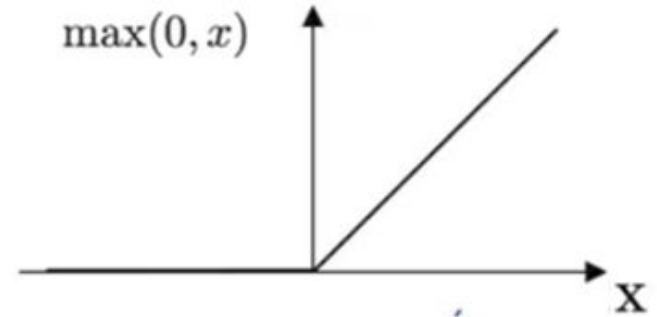
$$f(x) = \text{relu}(x) = \max(0, x)$$

- Non-linear transformation performed on data
- Output is continuous between

$$[0, +\infty]$$

- Mainly used for hidden layers

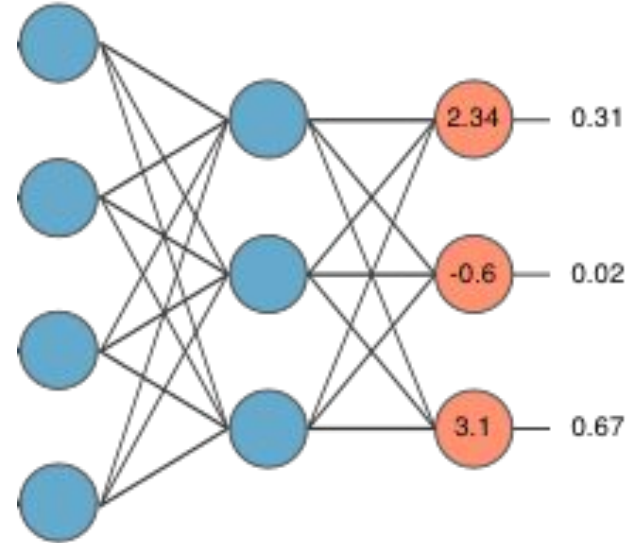
ReLU Function



Softmax Function

$$f(x_i) = \text{softmax}(x_i) = \frac{e^{x_i}}{\sum_n e^{x_n}}$$

- Non-linear transformation performed on data
- Output is continuous between $[0, 1]$
- Mainly used for the output layer in multi-class and multi-label classification



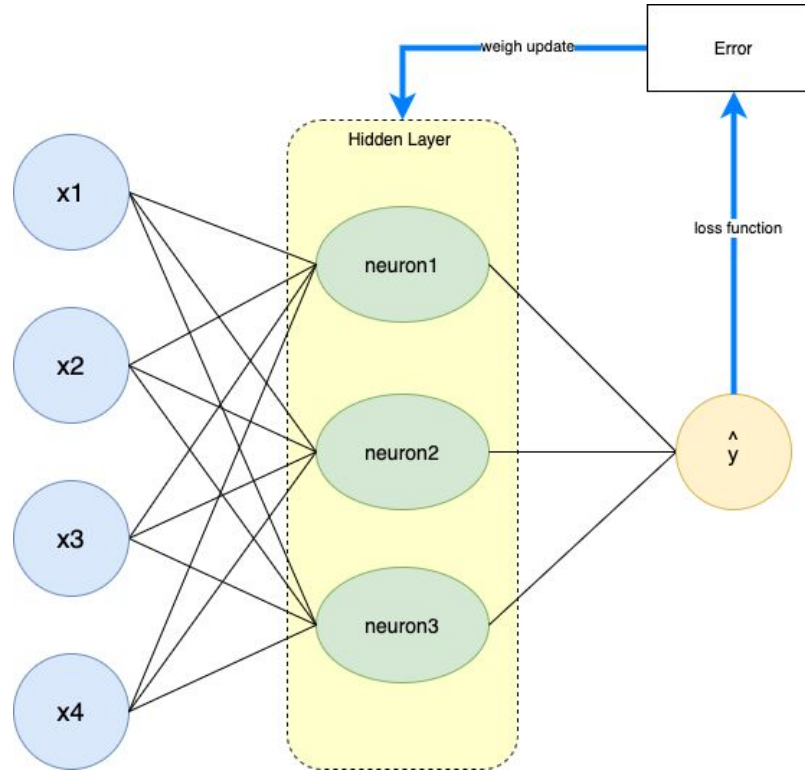


Back Propagation

Back Propagation

Forward propagation:

- Compute matrices multiplication and addition
- Apply activation function
- Make prediction



Back propagation:

- Compare prediction versus actual value with loss function (error)
- Using the error to update neurons weight

Weight Update

$$*W_x = W_x - \text{Learning rate} \left(\frac{\partial \text{Error}}{\partial W_x} \right)$$

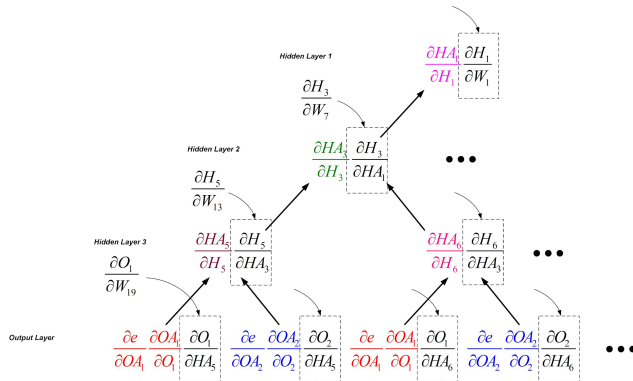
New weight $\uparrow$ W_x $\leftarrow$ Old weight
 Learning rate $\uparrow$ Learning rate
 Derivative of Error with respect to weight $\uparrow$ $\left(\frac{\partial \text{Error}}{\partial W_x} \right)$

Learning rate is the hyperparameter that will define how much the weights will be updated

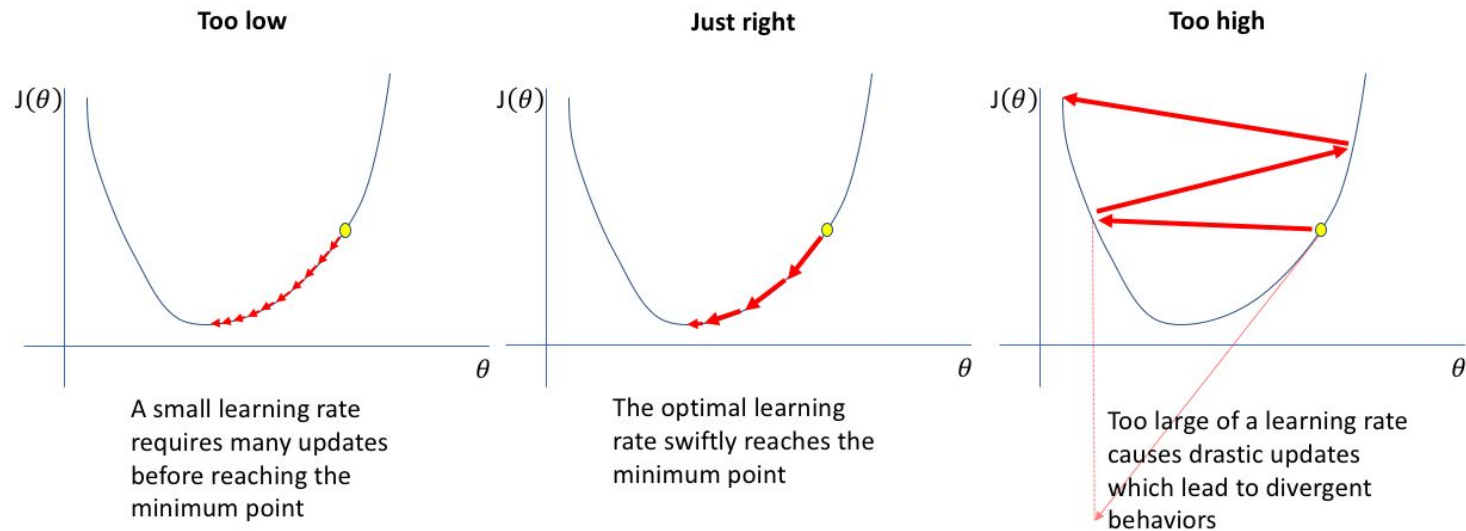
We calculate the gradient (differentiation) of a weight against the error. This will tell if we need to increase or decrease its value to get better prediction

Using the chain rule (calculus), we can move backward to calculate the gradient for each weight

We can repeat the process of applying forward and back propagation multiple times to find the best weights (Gradient Descent)



Learning Rate



Usually, start with 0.001

Epoch

Neural Networks is an optimisation algorithm. It doesn't try to find the best solution in one go. It keeps learning by adjusting its weights to get closer and closer to the solution.

It needs to go through the dataset multiple times

Epoch 

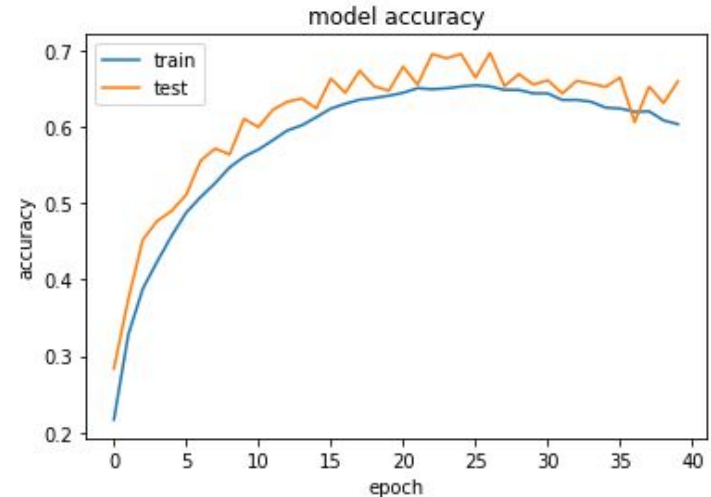
An Epoch represents one iteration over the entire dataset.

Batch 

We cannot pass the entire dataset into the neural network at once. So, we divide the dataset into number of batches.

Iteration 

If we have 10,000 images as data and a batch size of 200, then an epoch should contain $10,000/200 = 50$ iterations.



Optimisers

Batch Gradient Descent

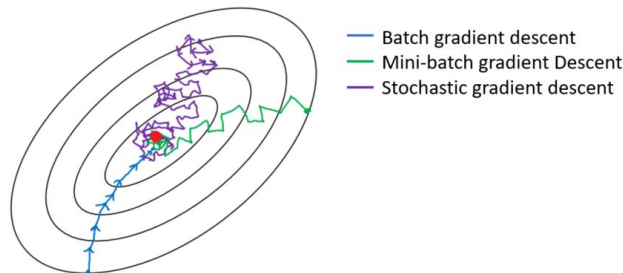
Gradient is calculated for all observations at once and weights are updated accordingly. Faster convergence but may be stuck before finding local minima

Stochastic Gradient Descent

Gradient is calculated for one single observation only. Higher chance to find the local minima but it is very slow

Mini-batch Gradient Descent

Gradient is calculated for a random sample of observations at once. Fast and good chance to find the local minima



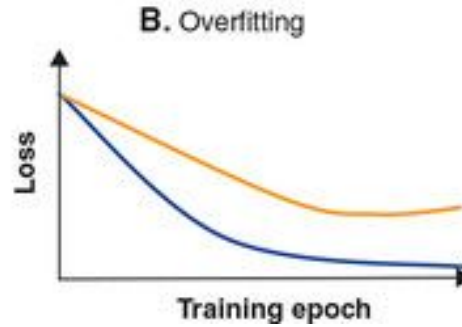
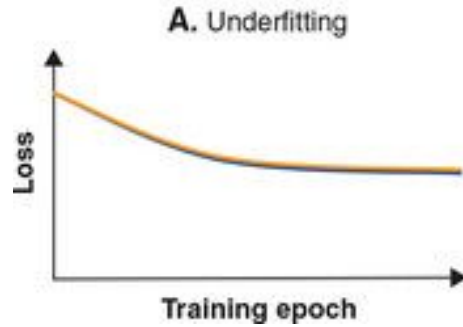
RMSprop

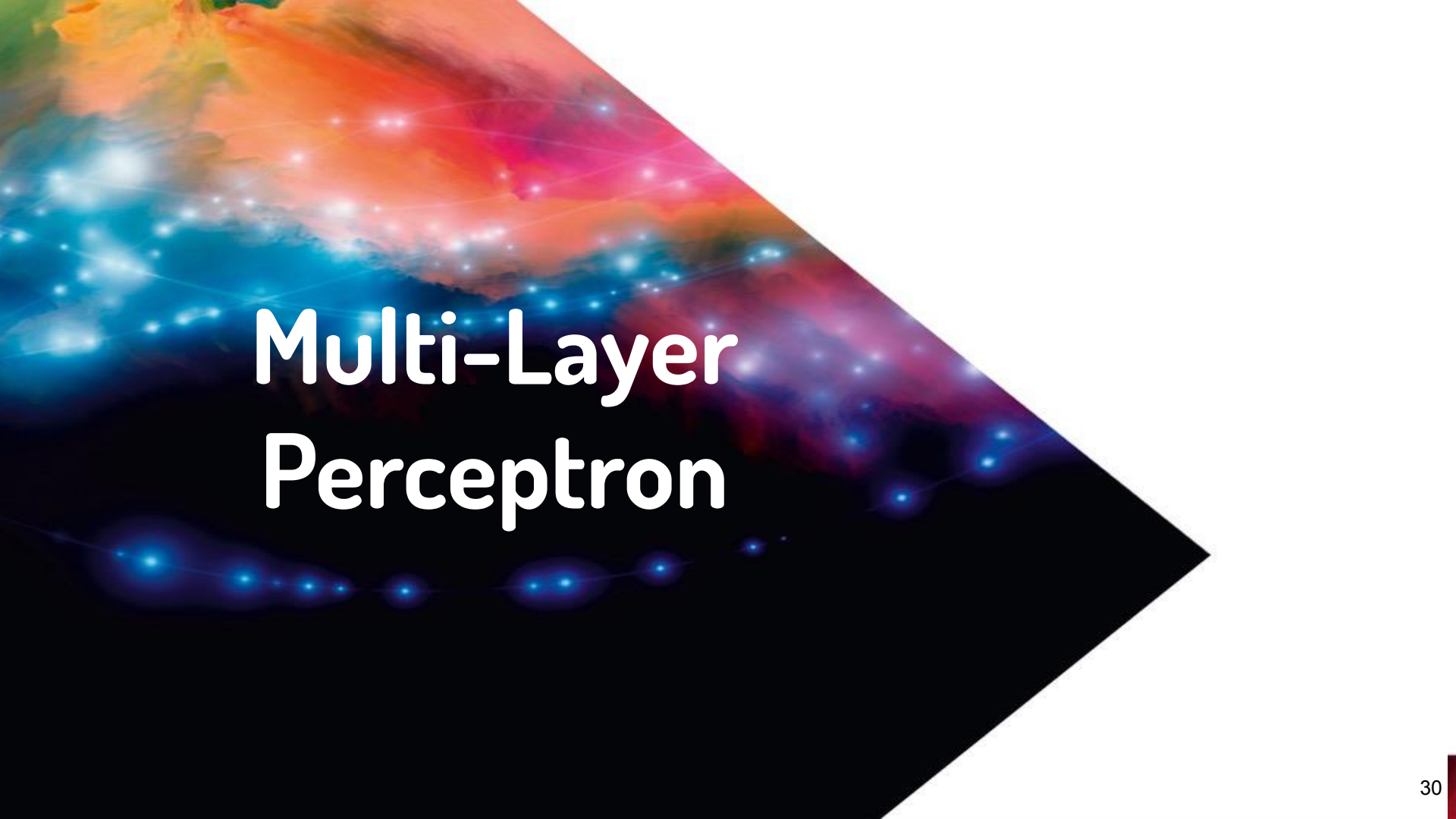
Diminishes learning rates by using a moving average of the squared gradient

ADAM

Apply momentum of first and second order. Bigger step for initial update then smaller step while getting closer to the local minima

Learning Process





Multi-Layer Perceptron

MLP Regressor

Generic API for sklearn models:



Import class

```
from sklearn.neural_network import MLPRegressor
```



Instantiate a model with hyperparameter

```
regr = MLPRegressor(random_state=1, max_iter=500)
```



Fit a model

```
regr.fit(X_train, y_train)
```

```
regr = MLPRegressor(random_state=1, max_iter=500).fit(X_train, y_train)
```



Predict with a model

```
regr.predict(X_test[:2])
```


MLP Classifier

Generic API for sklearn models:



Import class

```
from sklearn.neural_network import MLPClassifier
```



Instantiate a model with hyperparameter

```
clf = MLPClassifier(solver='lbfgs', alpha=1e-5,  
                    hidden_layer_sizes=(5, 2), random_state=1)
```



Fit a model

```
clf.fit(X, y)
```



Predict with a model

```
clf.predict([[2., 2.], [-1., -2.]])
```

Sklearn Parameters

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html#sklearn.neural_network.MLPRegressor
https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html#sklearn.neural_network.MLPClassifier

Parameters:

hidden_layer_sizes : *array-like of shape(n_layers - 2,)*, **default=(100,)**
The ith element represents the number of neurons in the ith hidden layer.

activation : *{'identity', 'logistic', 'tanh', 'relu'}*, **default='relu'**
Activation function for the hidden layer.

- 'identity', no-op activation, useful to implement linear bottleneck, returns $f(x) = x$
- 'logistic', the logistic sigmoid function, returns $f(x) = 1 / (1 + \exp(-x))$.
- 'tanh', the hyperbolic tan function, returns $f(x) = \tanh(x)$.
- 'relu', the rectified linear unit function, returns $f(x) = \max(0, x)$

solver : *{'lbfgs', 'sgd', 'adam'}*, **default='adam'**
The solver for weight optimization.

- 'lbfgs' is an optimizer in the family of quasi-Newton methods.
- 'sgd' refers to stochastic gradient descent.
- 'adam' refers to a stochastic gradient-based optimizer proposed by Kingma, Diederik, and Jimmy Ba

Note: The default solver 'adam' works pretty well on relatively large datasets (with thousands of training samples or more) in terms of both training time and validation score. For small datasets, however, 'lbfgs' can converge faster and perform better.

alpha : *float*, **default=0.0001**
Strength of the L2 regularization term. The L2 regularization term is divided by the sample size when added to the loss.

batch_size : *int*, **default='auto'**
Size of minibatches for stochastic optimizers. If the solver is 'lbfgs', the regressor will not use minibatch. When set to "auto", `batch_size=min(200, n_samples)`.

learning_rate : *{'constant', 'invscaling', 'adaptive'}*, **default='constant'**
Learning rate schedule for weight updates.

- 'constant' is a constant learning rate given by 'learning_rate_init'.
- 'invscaling' gradually decreases the learning rate 'learning_rate' at each time step 't' using an inverse scaling exponent of 'power_t'. $\text{effective_learning_rate} = \text{learning_rate_init} / \text{pow}(t, \text{power_t})$
- 'adaptive' keeps the learning rate constant to 'learning_rate_init' as long as training loss keeps decreasing. Each time two consecutive epochs fail to decrease training loss by at least tol, or fail to increase validation score by at least tol if 'early_stopping' is on, the current learning rate is divided by 5.

Only used when solver='sgd'.

learning_rate_init : *float*, **default=0.001**

The initial learning rate used. It controls the step-size in updating the weights. Only used when solver='sgd' or 'adam'.

power_t : *float*, **default=0.5**

The exponent for inverse scaling learning rate. It is used in updating effective learning rate when the learning_rate is set to 'invscaling'. Only used when solver='sgd'.

max_iter : *int*, **default=200**

Maximum number of iterations. The solver iterates until convergence (determined by 'tol') or this number of iterations. For stochastic solvers ('sgd', 'adam'), note that this determines the number of epochs (how many times each data point will be used), not the number of gradient steps.

shuffle : *bool*, **default=True**

Whether to shuffle samples in each iteration. Only used when solver='sgd' or 'adam'.

random_state : *int, RandomState instance*, **default=None**

Determines random number generation for weights and bias initialization, train-test split if early stopping is used, and batch sampling when solver='sgd' or 'adam'. Pass an int for reproducible results across multiple function calls. See Glossary.



Time for Lab

